

The resilience shield

HybridCache · fairness · background workers

TMS scale

```
await cache.GetOrCreateAsync(  
    $"course:{code}", code, factory...);  
  
AddTokenBucketLimiter("search", ...);  
  
return Accepted(statusUrl, body);
```

The survival strategy

Session goals

By the end of this session, you will:

- Implement HybridCache eliminate stampedes with atomic refetching
- Understand L1/L2 cache layers and when each TTL applies (multi-server drift)
- Enforce API fairness with the token-bucket rate limiter
- Handle long-running operations using 202 Accepted with BackgroundService
- Build thread-safe queues with Channel<T> between HTTP and background work

The cache stampede problem

- 1. Standard way: 20 people miss the same book all 20 run into the archive at once; the archive gets crushed
- 2. Atomic way (HybridCache): the librarian makes 19 wait at the desk, fetches once, shows all 20 the same copy

Atomic refetching

Name this out loud in the room: atomic refetching hallmark of senior-level scaling. Smart librarian = one in-flight refetch, many awaiters.

Demo cue

50 × GET /api/courses: IMemoryCache → 50 DB queries; HybridCache → 1 query make the log visceral.

HybridCache the one-query rule

Course read path

```
public async Task<CourseDto> GetCourseAsync(
    string code, CancellationToken ct)
{
    return await cache.GetOrCreateAsync(
        $"course:{code}",
        code, // state param avoids closure allocation
        async (courseCode, token) =>
            await repo.GetByCodeAsync(courseCode, token),
        new HybridCacheEntryOptions {
            Expiration = TimeSpan.FromMinutes(10), // L2 (Redis) TTL
            LocalCacheExpiration = TimeSpan.FromMinutes(2), // L1 in-memory
        }, tags: ["courses"], cancellation_token: ct);
}
```

- Atomic L1 layer blazing-fast in-memory lookup
- Auto L2 fallback plugs into Redis for multi-server deployments
- Tag-based invalidation flush all "courses" in one call after catalog changes

Fairness gate token bucket

Blocking the abuser a misconfigured script at 100 req/s earns 429; honest students stay fast.

Program.cs

```
builder.Services.AddRateLimiter(options =>
{
    options.AddTokenBucketLimiter("search", opt =>
    {
        opt.TokenLimit = 10;

        opt.ReplenishmentPeriod = TimeSpan.FromSeconds(10);

        opt.TokensPerPeriod = 5;

        opt.QueueLimit = 2;
    });
});
```

A buggy script must not starve honest students 429 Too Many Requests is fairness, not punishment.

Background workers kitchen pager (202 Accepted)

Controller

```
[HttpPost("transcripts")]

public async Task<IActionResult> RequestTranscript(
    TranscriptRequest request, CancellationToken ct)
{
    var reportId = Guid.NewGuid().ToString("N")[..8];

    await channel.Writer.WriteAsync(
        request.WithReportId(reportId), ct);

    return Accepted(
        Url.Action(nameof(GetStatus), new { id = reportId }),
        new { ReportId = reportId, Status = "Queued" });
}
```

- Transcript generation (~15s) do not block the HTTP thread on Kestrel
- Heavy work runs in BackgroundService + Channel<T>; client polls the status URL from Accepted
- 202 Accepted + location enterprise pattern for PDFs, exports, batch imports (interview-ready)

Lab brief Session 2

- M7-Lab-Workbook Exercises 3, 4, and 5
- Exercise 3: HybridCache stampede protection, tag invalidation
- Exercise 4: Rate limiting token bucket, 429 responses
- Exercise 5: Background workers BackgroundService, Channel<T>, 202 Accepted
- Checkpoints: 50 concurrent reads → 1 DB query; burst → 429 + Retry-After; POST returns 202 immediately

Session 2 summary

Interviewer: "How do you handle a popular endpoint during a traffic spike?" Answer: "I use HybridCache with atomic refetching. Only one request hits the database while others wait asynchronously, preventing a stampede."

- HybridCache production standard for surviving traffic spikes (stampede protection)
- Rate limiting protects infrastructure from abuse and buggy clients
- BackgroundService + Channel<T> long-running work without blocking HTTP threads
- 202 Accepted professional pattern for deferred operations

Next: Session 3 efficiency & real-time

Data shaping with ?fields=, HATEOAS navigation on v2, SignalR hubs with typed clients and course-group dispatch for TMS notifications.